

```
$ lab --phase 0 --num 2
```

Lab 2

Bash Monitoring Script

Procédure complète — Script de monitoring serveur

Bash	Cron	JSON	Alertes
------	------	------	---------

Système	Auteur	Date	Statut
Ubuntu Server 22.04	Nathan	Juin 2026	Validé ✓

CHAPITRE 01 Squelette du script

Créer le fichier du script et configurer les droits nécessaires.

1.1 Prérequis — Fichier log

```
• • • bash
# Créer le fichier log avec les bons droits
sudo touch /var/log/monitor.log
sudo chown user:user /var/log/monitor.log
```

■ ■ Erreur rencontrée

Sans chown, le script échoue sur 'tee: Permission denied'.

L'utilisateur courant ne peut pas écrire dans /var/log/ sans droits explicites.

1.2 Créer monitor.sh

```
• • • bash
nano ~/lab2-bash-monitoring/monitor.sh
```

```
#!/bin/bash

# ████████████████████████████████████████████████████████████

# monitor.sh – Monitoring serveur Linux

# Usage : ./monitor.sh

# ████████████████████████████████████████████████████████████

set -euo pipefail


# ███ Constantes ████████████████████████████████████████████████████████████

LOG="/var/log/monitor.log"
RAPPORT="/tmp/health_report.json"
SEUIL_DISK=80
SEUIL_DISK_CRIT=90
SEUIL_RAM=85
SEUIL_HTTP=500
SERVICES=("ssh" "ufw" "fail2ban" "log-python")


# ███ Couleurs terminal ████████████████████████████████████████████████████████████

OK="\e[32m[OK]\e[0m"
WARN="\e[33m[WARN]\e[0m"
CRIT="\e[31m[CRIT]\e[0m"


# ███ Fonction log ████████████████████████████████████████████████████████████

log() {
    local niveau=$1
    local message=$2
    local ts=$(date '+%Y-%m-%d %H:%M:%S')
    echo "[${ts}] [${niveau}] $message" | tee -a "$LOG"
}

log "INFO" "=== Début monitoring $(hostname) ==="
```

CHAPITRE 02

Fonctions de vérification

2.1 check_disk() — Espace disque

```
check_disk() {  
    local usage  
    usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')  
    if [ "$usage" -ge "$SEUIL_DISK_CRIT" ]; then  
        log "CRITICAL" "Disque a ${usage}%"  
        echo -e "$CRIT Disque : ${usage}%"  
        return 2  
    elif [ "$usage" -ge "$SEUIL_DISK" ]; then  
        log "WARNING" "Disque a ${usage}%"  
        echo -e "$WARN Disque : ${usage}%"  
        return 1  
    else  
        log "INFO" "Disque OK : ${usage}%"  
        echo -e "$OK Disque : ${usage}%"  
        return 0  
    fi  
}  
  
check_disk
```

2.2 check_ram() — Mémoire RAM

```
check_ram() {  
    local total used usage  
    total=$(free | awk 'NR==2 {print $2}')  
    used=$(free | awk 'NR==2 {print $3}')  
    usage=$(( used * 100 / total ))  
    if [ "$usage" -ge "$SEUIL_RAM" ]; then  
        log "WARNING" "RAM a ${usage}%"  
        echo -e "$WARN RAM : ${usage}%"  
        return 1  
    else  
        log "INFO" "RAM OK : ${usage}%"  
        echo -e "$OK RAM : ${usage}%"  
        return 0  
    fi  
}  
  
check_ram
```

2.3 check_service() — Services systemd

```
bash
check_service() {
    local service=$1
    if systemctl is-active --quiet "$service"; then
        log "INFO" "Service $service : actif"
        echo -e "$OK Service : $service"
        return 0
    else
        log "CRITICAL" "Service $service : INACTIF"
        echo -e "$CRIT Service : $service INACTIF"
        return 1
    fi
}

for service in "${SERVICES[@]}; do
    check_service "$service" || true
done
```

2.4 check_http() — Disponibilité HTTP

```
bash
check_http() {
    local url=$1
    local start end duration code
    start=$(date +%s%3N)
    code=$(curl -o /dev/null -s -w "%{http_code}" \
        --max-time 5 "$url")
    end=$(date +%s%3N)
    duration=$(( end - start ))
    if [ "$code" -eq 000 ]; then
        log "CRITICAL" "HTTP $url : injoignable"
        echo -e "$CRIT HTTP $url : injoignable"
        return 2
    elif [ "$code" -ne 200 ]; then
        log "WARNING" "HTTP $url : code $code"
        echo -e "$WARN HTTP $url : code $code"
        return 1
    elif [ "$duration" -gt "$SEUIL_HTTP" ]; then
        log "WARNING" "HTTP $url : lent (${duration}ms)"
        echo -e "$WARN HTTP $url : ${duration}ms"
        return 1
    else
        log "INFO" "HTTP $url : OK (${duration}ms)"
        echo -e "$OK HTTP $url : ${duration}ms"
        return 0
    fi
}

check_http "http://localhost" || true
```

CHAPITRE 03

Rapport JSON

```
bash
generer_rapport() {
    local disk ram
    disk=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
    ram=$(free | awk \
        'NR==2 {used=$3; total=$2; print int(used*100/total)}')

    cat > "$RAPPOR" << EOF
{
    "date": "$(date '+%Y-%m-%d %H:%M:%S')",
    "hostname": "$(hostname)",
    "disque_pct": $disk,
    "ram_pct": $ram,
    "services": {
        "ssh": "$(systemctl is-active ssh)",
        "ufw": "$(systemctl is-active ufw)",
        "fail2ban": "$(systemctl is-active fail2ban)",
        "log-python": "$(systemctl is-active log-python)"
    }
}
EOF
    log "INFO" "Rapport JSON genere : $RAPPOR"
}

generer_rapport
log "INFO" "=== Fin monitoring ==="
```

Résultat dans /tmp/health_report.json :

```
json
{
    "date": "2026-06-20 15:23:00",
    "hostname": "Serveur-Linux",
    "disque_pct": 46,
    "ram_pct": 6,
    "services": {
        "ssh": "active",
        "ufw": "active",
        "fail2ban": "active",
        "log-python": "active"
    }
}
```

CHAPITRE 04

Configuration Cron

```
# Ouvrir l'éditeur cron
crontab -e

# Ajouter cette ligne à la fin du fichier
*/5 * * * * /bin/bash /home/user/lab2-bash-monitoring/monitor.sh

# Vérifier que le cron est enregistré
crontab -l | grep monitor
```

CHAPITRE 05

Validation finale

✓	Squelette	set -euo pipefail, constantes configurables, fonction log
✓	check_disk	WARNING à 80%, CRITICAL à 90%
✓	check_ram	WARNING à 85%
✓	check_service	Vérifie ssh, ufw, fail2ban, log-python
✓	check_http	Test localhost, timeout 5s, seuil 500ms
✓	Rapport JSON	Généré dans /tmp/health_report.json
✓	Cron	Exécution automatique toutes les 5 minutes